

MANUAL TÉCNICO – Sistema INSAVA01



1.1 Nombre del sistema

INSAVA01 – Sistema de Ventas, Compras e Inventario

1.2 Propósito del documento

Describir la estructura interna, componentes técnicos, arquitectura, dependencias, instalación, configuración y funcionamiento interno del sistema para técnicos y desarrolladores.

1.3 Público objetivo

- Administradores del sistema
 - Técnicos de soporte
 - Programadores
- Ingenieros de implementación

Contents

2. Arquitectura del sistema	4
2.1 Arquitectura general	4
2.2 Patrones de desarrollo	4
2.3 Tecnologías utilizadas	4
3. Estructura del proyecto	5
3.1 Carpetas principales	5
4. Base de datos	6
4.1 Modelo de datos	6
4.2 Relaciones principales	6
4.3 Script de creación	7
5. Instalación del sistema	28
5.1 Requisitos previos	28
5.2 Procedimiento	28
6. Configuración técnica	29
6.1 Cadena de conexión	29
6.2 Configuración de reportes	29
6.3 Configuración de roles y usuarios	29
7. Descripción técnica por módulos	30
7.1 Módulo de Clientes	30
7.2 Módulo de Proveedores	30
7.3 Módulo de Productos	30
7.4 Módulo de Compras	30
7.5 Módulo de Ventas	31
7.6 Módulo de Kardex	31
7.7 Módulo de Usuarios	31
7.8 Configuración de Empresa	31
7.9 Descripción de Clases del Sistema INSAVA01	32
8. Seguridad	35
8.1 Seguridad de acceso	35
8.2 Seguridad de BD	35
9. Mantenimiento técnico	36

9.1 Tareas periódicas	36
9.2 Respaldo	36
10. Troubleshooting técnico	37

2. Arquitectura del sistema

2.1 Arquitectura general

El sistema INSAVA01 está compuesto por:

- **Aplicación de escritorio Windows (WinForms / C#)**
- **Base de datos SQL Server**
- **Reportes generados con ReportViewer**
- **Interfaz modular** basada en formularios (Customers, Products, Sales, etc.)

2.2 Patrones de desarrollo

- Arquitectura por **capas simples**:
 - **Presentación** (WinForms)
 - **Negocio** (clases para validación, cálculos y lógica)
 - **Datos** (conexiones y consultas SQL)
- Conexión a BD mediante **SqlConnection, SqlCommand, SqlDataAdapter**.

2.3 Tecnologías utilizadas

Componente	Tecnología
Lenguaje	C# (.NET Framework)
Interfaz	Windows Forms
Reportes	ReportViewer
Base de datos	SQL Server
Almacenamiento	Archivos locales y BD
Framework	.NET Framework 4+

3. Estructura del proyecto

3.1 Carpetas principales

Dentro de la solución (INSAVA01) se encuentran típicamente:

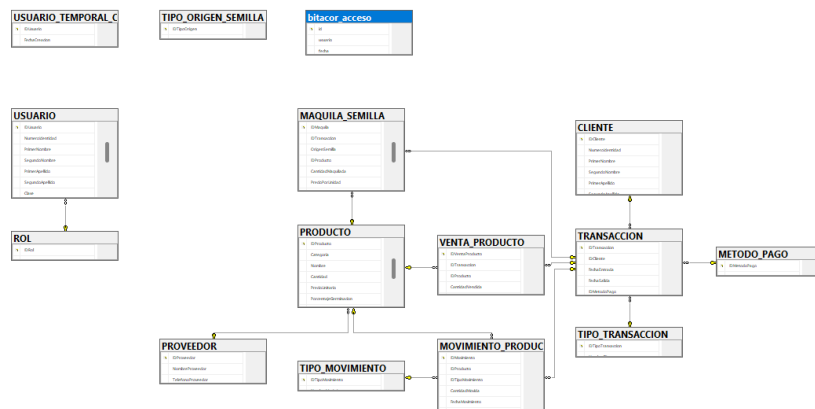
- **/Forms** → Formularios principales (Clientes, Proveedores, Productos, Ventas, etc.)
- **/Reports** → Archivos .rdlc
- **/Data** → Conexiones, consultas y clases de acceso a datos
- **/Models** → Clases de entidades (Producto, Cliente, Usuario...)
- **/bin/Debug** → Ejecutable y dependencias
- **/Properties** → Configuración del proyecto

4. Base de datos

4.1 Modelo de datos

Tablas esenciales:

- **Clientes**
- **Proveedores**
- **Productos**
- **Compras**
- **DetalleCompra**
- **Ventas**
- **DetalleVenta**
- **Usuarios**
- **Empresa**
- **Kardex**



4.2 Relaciones principales

- Un **cliente** → muchas **ventas**
- Un **proveedor** → muchas **compras**
- Un **producto** → muchos movimientos en **kardex**

4.3 Script de creación

-- SCRIPT COMPLETO FINAL: SISTEMASEMILLA CORREGIDO

-- Incluye usuario temporal que expira en 1 hora

USE master;

GO

DROP DATABASE IF EXISTS SISTEMASEMILLA;

GO

CREATE DATABASE SISTEMASEMILLA;

GO

USE SISTEMASEMILLA;

GO

CREATE TABLE bitacora_accesos (
 id INT IDENTITY(1,1) PRIMARY KEY,
 usuario VARCHAR(100) NOT NULL,
 fecha_hora DATETIME NOT NULL,
 status_usuario VARCHAR(10) NOT NULL,
 intento_exitoso BIT NOT NULL
);

-- 1. CATÁLOGOS BÁSICOS

CREATE TABLE ROL (
 IDRol INT IDENTITY(1,1) PRIMARY KEY,
 NombreRol VARCHAR(50) NOT NULL UNIQUE

```
);
```

```
GO
```

```
CREATE TABLE METODO_PAGO (
```

```
    IDMetodoPago INT IDENTITY(1,1) PRIMARY KEY,
```

```
    NombreMetodo VARCHAR(50) NOT NULL
```

```
);
```

```
GO
```

```
CREATE TABLE TIPO_TRANSACCION (
```

```
    IDTipoTransaccion INT IDENTITY(1,1) PRIMARY KEY,
```

```
    NombreTipo VARCHAR(100) NOT NULL
```

```
);
```

```
GO
```

```
CREATE TABLE TIPO_MOVIMIENTO (
```

```
    IDTipoMovimiento INT IDENTITY(1,1) PRIMARY KEY,
```

```
    NombreMovimiento VARCHAR(50) NOT NULL
```

```
);
```

```
GO
```

```
CREATE TABLE TIPO_ORIGEN_SEMILLA (
```

```
    IDTipoOrigen INT IDENTITY(1,1) PRIMARY KEY,
```

```
    NombreOrigen VARCHAR(50) NOT NULL
```

```
);
```

```
GO
```

```
-----  
-- 2. ENTIDADES PRINCIPALES  
-----
```



```
CREATE TABLE CLIENTE (  
    IDCliente INT IDENTITY(1,1) PRIMARY KEY NOT NULL,  
    NumeroIdentidad VARCHAR(50) NOT NULL,  
    PrimerNombre VARCHAR(50) NOT NULL,  
    SegundoNombre VARCHAR(50),  
    PrimerApellido VARCHAR(50) NOT NULL,  
    SegundoApellido VARCHAR(50),  
    NumTel VARCHAR(15) NOT NULL,  
    Activo BIT NOT NULL DEFAULT 1  
);  
GO
```

```
CREATE TABLE PROVEEDOR (  
    IDProveedor INT IDENTITY(1,1) PRIMARY KEY,  
    NombreProveedor VARCHAR(100) NOT NULL,  
    TelefonoProveedor VARCHAR(50) NOT NULL,  
    Activo BIT NOT NULL DEFAULT 1  
);  
GO
```

```
CREATE TABLE USUARIO (  
    IDUsuario INT IDENTITY(1,1) PRIMARY KEY,  
    NumeroIdentidad VARCHAR(15) NOT NULL UNIQUE,  
    PrimerNombre VARCHAR(50) NOT NULL,  
    SegundoNombre VARCHAR(50),  
    PrimerApellido VARCHAR(50) NOT NULL,  
    SegundoApellido VARCHAR(50),  
    Clave VARCHAR(100) NOT NULL,  
    IDRol INT NOT NULL,  
    Activo BIT NOT NULL DEFAULT 1,
```

```

        FechaExpiracion DATETIME NULL,
CONSTRAINT FK_Usuario_Rol FOREIGN KEY (IDRol)
REFERENCES ROL(IDRol) ON DELETE CASCADE,

);
GO

```

```

CREATE TABLE PRODUCTO (
    IDProducto INT IDENTITY(1,1) PRIMARY KEY,
    Categoria VARCHAR(50) NOT NULL,
    Nombre VARCHAR(100) NOT NULL,
    Cantidad DECIMAL(10,2) NOT NULL DEFAULT 0,
    PrecioUnitario DECIMAL(10,2) NOT NULL DEFAULT 0,
    PorcentajeGerminacion DECIMAL(4,2) CHECK (PorcentajeGerminacion BETWEEN 0
AND 1),
    PrecioMaquila DECIMAL(10,2) NULL,
    IDProveedor INT NULL,
    Activo BIT NOT NULL DEFAULT 1,
    CONSTRAINT FK_Producto_Proveedor FOREIGN KEY (IDProveedor)
REFERENCES PROVEEDOR(IDProveedor),
    CONSTRAINT CK_Producto_PorcentajeGerminacion_Semilla
CHECK (
        (Categoria IN ('Semilla', 'Semilla Maquilada') AND PorcentajeGerminacion IS NOT
NULL)
        OR
        (Categoria NOT IN ('Semilla', 'Semilla Maquilada') AND PorcentajeGerminacion IS
NULL)
    ),
    CONSTRAINT CK_Producto_PrecioMaquila_Semilla
CHECK (
        (Categoria IN ('Semilla', 'Semilla Maquilada') AND PrecioMaquila IS NOT NULL)

```

```

        OR
        (Categoria NOT IN ('Semilla', 'Semilla Maquilada') AND PrecioMaquila IS NULL)
    )
);
GO

```

```

-----
-- 3. TABLAS DE OPERACIONES
-----

```

```

CREATE TABLE TRANSACCION (
    IDTransaccion INT IDENTITY(1,1) PRIMARY KEY,
    IDCliente INT NOT NULL,
    FechaEntrada DATETIME NOT NULL,
    FechaSalida DATETIME NULL,
    IDMetodoPago INT NULL,
    IDTipoTransaccion INT NOT NULL,
    Activo BIT NOT NULL DEFAULT 1,
    CONSTRAINT FK_Trans_Cliente FOREIGN KEY (IDCliente) REFERENCES
CLIENTE(IDCliente),
    CONSTRAINT FK_Trans_Metodo FOREIGN KEY (IDMetodoPago)
    REFERENCES METODO_PAGO(IDMetodoPago),
    CONSTRAINT FK_Trans_Tipo FOREIGN KEY (IDTipoTransaccion)
    REFERENCES TIPO_TRANSACCION(IDTipoTransaccion)
);
GO

```

```

CREATE TABLE MOVIMIENTO_PRODUCTO (
    IDMovimiento INT IDENTITY(1,1) PRIMARY KEY,
    IDProducto INT NOT NULL,
    IDTipoMovimiento INT NOT NULL,

```

```

CantidadMovida DECIMAL(10,2) NOT NULL,
FechaMovimiento DATETIME NOT NULL DEFAULT GETDATE(),
Descripcion VARCHAR(MAX),
IDTransaccion INT NULL,
Activo BIT NOT NULL DEFAULT 1,
CONSTRAINT FK_MovProd_Prod FOREIGN KEY (IDProducto)
    REFERENCES PRODUCTO(IDProducto),
CONSTRAINT FK_MovProd_TipoMov FOREIGN KEY (IDTipoMovimiento)
    REFERENCES TIPO_MOVIMIENTO(IDTipoMovimiento),
CONSTRAINT FK_MovProd_Trans FOREIGN KEY (IDTransaccion)
    REFERENCES TRANSACCION(IDTransaccion)
);
GO

```

```

CREATE TABLE VENTA_PRODUCTO (
    IDVentaProducto INT IDENTITY(1,1) PRIMARY KEY,
    IDTransaccion INT NOT NULL,
    IDProducto INT NOT NULL,
    CantidadVendida DECIMAL(10,2) NOT NULL,
    TotalVenta DECIMAL(10,2) NOT NULL,
    Activo BIT NOT NULL DEFAULT 1,
    CONSTRAINT FK_VentaProd_Trans FOREIGN KEY (IDTransaccion)
        REFERENCES TRANSACCION(IDTransaccion),
    CONSTRAINT FK_VentaProd_Prod FOREIGN KEY (IDProducto)
        REFERENCES PRODUCTO(IDProducto)
);
GO

```

```

CREATE TABLE MAQUILA_SEMILLA (
    IDMaquila INT IDENTITY(1,1) PRIMARY KEY,

```

```

IDTransaccion INT NOT NULL,

OrigenSemilla VARCHAR(20) NOT NULL DEFAULT 'Cliente' CHECK (OrigenSemilla IN
('Cliente', 'Inventario')),

IDProducto INT NULL,

CantidadMaquilada INT NOT NULL,

PrecioPorUnidad DECIMAL(10,2) NOT NULL,

FechaInicio DATE NOT NULL,

FechaEntrega DATE NOT NULL,

Activo BIT NOT NULL DEFAULT 1,

CONSTRAINT FK_Maquila_Trans FOREIGN KEY (IDTransaccion)

REFERENCES TRANSACCION(IDTransaccion),

CONSTRAINT FK_Maquila_Prod FOREIGN KEY (IDProducto)

REFERENCES PRODUCTO(IDProducto)

);

GO

```

```

-----

-- 4. TRIGGER MAQUILA

-----

```

```

CREATE TRIGGER trg_InsertMaquilaSemilla

ON MAQUILA_SEMILLA

INSTEAD OF INSERT

AS

BEGIN

INSERT INTO MAQUILA_SEMILLA (

IDTransaccion,

OrigenSemilla,

IDProducto,

CantidadMaquilada,

PrecioPorUnidad,

```

```

        FechaInicio,
        FechaEntrega
    )
SELECT
    IDTransaccion,
    OrigenSemilla,
    IDProducto,
    CantidadMaquilada,
    PrecioPorUnidad,
    FechaInicio,
    DATEADD(DAY, 30, FechaInicio)
FROM inserted;
END;
GO

```

-- 5. VISTAS

```
CREATE VIEW VISTAFUSUARIO AS
```

```
SELECT
```

```

    u.IDUsuario,
    u.Numeroidentidad,
    (u.PrimerNombre + ' ' + ISNULL(u.SegundoNombre,'') + ' ' + u.PrimerApellido + ' ' +
    ISNULL(u.SegundoApellido,'')) AS NombreCompleto,
    u.Clave,
    r.NombreRol,
    u.Activo

```

```
FROM USUARIO u
```

```
INNER JOIN ROL r ON u.IDRol = r.IDRol;
```

```
GO
```

```
CREATE VIEW VISTAFCLIENTE AS
```

```
SELECT
```

```
    c.IDCliente,
```

```
    c.Numeroidentidad,
```

```
    (c.PrimerNombre + ' ' + ISNULL(c.SegundoNombre,'') + ' ' + c.PrimerApellido + ' ' +  
    ISNULL(c.SegundoApellido,'')) AS NombreCompleto,
```

```
    c.NumTel AS Telefono,
```

```
    c.Activo
```

```
FROM CLIENTE c;
```

```
GO
```

```
CREATE VIEW VISTAPRODUCTOS AS
```

```
SELECT
```

```
    IDProducto,
```

```
    Categoria,
```

```
    CASE WHEN Categoria IN ('Semilla','Semilla Maquilada')
```

```
        THEN Nombre + ' (' + CAST(CAST(ISNULL(PorcentajeGerminacion*100,0) AS  
DECIMAL(5,2)) AS VARCHAR(6)) + '% Germinación'
```

```
        ELSE Nombre END AS Producto,
```

```
    CASE WHEN Categoria IN ('Semilla','Semilla Maquilada') THEN  
CAST(ISNULL(PorcentajeGerminacion*100,0) AS DECIMAL(5,2)) ELSE NULL END AS  
PorcentajeGerminacion,
```

```
    PrecioUnitario,
```

```
    PrecioMaquila
```

```
FROM PRODUCTO
```

```
GO
```

```
CREATE VIEW VISTAFactura AS
```

```
SELECT
```

```
    T.IDTransaccion AS NumeroFactura,
```

```

T.FechaEntrada AS FechaFactura,

(C.PrimerNombre + ' ' + ISNULL(C.SegundoNombre, '') + ' ' + C.PrimerApellido + ' ' +
ISNULL(C.SegundoApellido, '')) AS NombreCompletoCliente,

TT.NombreTipo AS TipoTransaccion,

MP.NombreMetodo AS MetodoPago,

VP.IDVentaProducto AS IDDetalleVenta, -- Identificador único para cada línea de detalle
de venta

VP.IDProducto,

VP_VIEW.Producto AS NombreProductoDetalle, -- Nombre del producto (incluye % de
germinación para semillas)

VP.CantidadVendida AS Cantidad,

P.PrecioUnitario AS PrecioUnitarioVenta, -- Precio del producto en el momento de la
venta

(VP.CantidadVendida * P.PrecioUnitario) AS SubtotalLinea, -- Subtotal por esta línea de
la factura

VP.Activo AS DetalleVentaActivo

FROM

TRANSACCION T

INNER JOIN CLIENTE C ON T.IDCliente = C.IDCliente

LEFT JOIN METODO_PAGO MP ON T.IDMetodoPago = MP.IDMetodoPago -- Se usa LEFT
JOIN para incluir transacciones que aún no tienen un método de pago asignado.

INNER JOIN TIPO_TRANSACCION TT ON T.IDTipoTransaccion = TT.IDTipoTransaccion

INNER JOIN VENTA_PRODUCTO VP ON T.IDTransaccion = VP.IDTransaccion

INNER JOIN VISTAPRODUCTOS VP_VIEW ON VP.IDProducto = VP_VIEW.IDProducto

inner join PRODUCTO P ON P.IDProducto = VP_VIEW.IDProducto-- Se une con
VISTAPRODUCTOS para obtener el nombre formateado del producto.

WHERE

TT.NombreTipo = 'Venta de producto'; -- Se filtra para mostrar solo transacciones de
venta.

GO

--VISTAS PARA REPORTERIA/BUSCADOR

```



```
CREATE VIEW VISTA_CLIENTE_SIMPLE AS
```

```
SELECT
```

```
(c.PrimerNombre + ' ' + ISNULL(c.SegundoNombre,'') + ' ' + c.PrimerApellido + ' ' +  
ISNULL(c.SegundoApellido,'')) AS NombreCompleto,
```

```
c.NumeroIdentidad AS Identidad,
```

```
c.NumTel AS Telefono
```

```
FROM CLIENTE c;
```

```
GO
```

```
CREATE VIEW VISTA_TRANSACCION_SIMPLE AS
```

```
SELECT
```

```
'T-' + RIGHT('000000' + CAST(t.IDTransaccion AS VARCHAR(10)),6) AS  
NumeroTransaccion,
```

```
CONVERT(VARCHAR(16), t.FechaEntrada, 120) AS FechaEntrada,
```

```
CONVERT(VARCHAR(16), t.FechaSalida, 120) AS FechaSalida,
```

```
(c.PrimerNombre + ' ' + ISNULL(c.SegundoNombre,'') + ' ' + c.PrimerApellido + ' ' +  
ISNULL(c.SegundoApellido,'')) AS NombreCliente,
```

```
tt.NombreTipo AS TipoTransaccion,
```

```
mp.NombreMetodo AS MetodoPago,
```

```
ISNULL(SUM(vp.TotalVenta),0) AS MontoTotal,
```

```
COUNT(vp.IDVentaProducto) AS Lineas
```

```
FROM TRANSACCION t
```

```
LEFT JOIN CLIENTE c ON t.IDCliente = c.IDCliente
```

```
LEFT JOIN TIPO_TRANSACCION tt ON t.IDTipoTransaccion = tt.IDTipoTransaccion
```

```
LEFT JOIN METODO_PAGO mp ON t.IDMetodoPago = mp.IDMetodoPago
```

```
LEFT JOIN VENTA_PRODUCTO vp ON vp.IDTransaccion = t.IDTransaccion AND vp.Activo =  
1
```

```
GROUP BY
```

```
t.IDTransaccion, t.FechaEntrada, t.FechaSalida,
```

```
c.PrimerNombre, c.SegundoNombre, c.PrimerApellido, c.SegundoApellido,
```

tt.NombreTipo, mp.NombreMetodo, t.Activo;

GO

CREATE VIEW VISTA_VENTA_SIMPLE AS

SELECT

p.Nombre AS NombreProducto,

p.Categoria,

vp.CantidadVendida,

vp.TotalVenta

FROM VENTA_PRODUCTO vp

LEFT JOIN PRODUCTO p ON vp.IDProducto = p.IDProducto;

GO

CREATE VIEW VISTA_MOVIMIENTO_SIMPLE AS

SELECT

CONVERT(VARCHAR(19), m.FechaMovimiento, 120) AS FechaMovimiento,

p.Nombre AS NombreProducto,

tm.NombreMovimiento AS TipoMovimiento,

m.CantidadMovida,

ISNULL(m.Descripcion, '') AS Descripcion,

CASE WHEN m.IDTransaccion IS NOT NULL THEN 'T-' + RIGHT('000000' +
CAST(m.IDTransaccion AS VARCHAR(10)), 6) ELSE '' END AS TransaccionRef

FROM MOVIMIENTO_PRODUCTO m

LEFT JOIN PRODUCTO p ON m.IDProducto = p.IDProducto

LEFT JOIN TIPO_MOVIMIENTO tm ON m.IDTipoMovimiento = tm.IDTipoMovimiento;

GO

CREATE VIEW VISTA_INVENTARIO_SIMPLE AS

SELECT

p.Nombre AS Producto,

```

p.Categoria,

p.Cantidad AS StockActual,

p.PrecioUnitario

FROM PRODUCTO p

WHERE p.Activo = 1;

GO


CREATE VIEW VISTA_MAQUILA_ESTADO_DETALLE AS

SELECT

'M-' + RIGHT('000000' + CAST(m.IDMaquila AS VARCHAR(10)),6) AS NumeroMaquila,

(c.PrimerNombre + ' ' + ISNULL(c.SegundoNombre,'') + ' ' + c.PrimerApellido + ' ' +

ISNULL(c.SegundoApellido,'')) AS Cliente,

CASE WHEN m.OrigenSemilla IS NULL OR LTRIM(RTRIM(m.OrigenSemilla)) = '' THEN

'Cliente' ELSE m.OrigenSemilla END AS OrigenSemilla,

ISNULL(p.Nombre,'(Sin producto)') AS Producto,

m.CantidadMaquilada,

ROUND(m.PrecioPorUnidad,2) AS PrecioPorUnidad,

ROUND(m.CantidadMaquilada * m.PrecioPorUnidad,2) AS MontoMaquila,

CONVERT(VARCHAR(10), m.FechaInicio, 120) AS FechaInicio,

CONVERT(VARCHAR(10), m.FechaEntrega, 120) AS FechaEntrega,

DATEDIFF(DAY, CAST(GETDATE() AS DATE), m.FechaEntrega) AS DiasRestantes,

CASE

WHEN m.Activo = 0 THEN 'Desactivada manualmente'

WHEN CAST(GETDATE() AS DATE) < m.FechaInicio THEN 'Programada'

WHEN CAST(GETDATE() AS DATE) BETWEEN m.FechaInicio AND m.FechaEntrega THEN

'Activa'

WHEN CAST(GETDATE() AS DATE) > m.FechaEntrega THEN 'Vencida'

ELSE 'Desconocido'

END AS EstadoMaquila,

CASE WHEN t.IDTransaccion IS NOT NULL THEN 'T-' + RIGHT('000000' +

CAST(t.IDTransaccion AS VARCHAR(10)) ,6) ELSE NULL END AS TransaccionAsociada

```

```

FROM MAQUILA_SEMILLA m
LEFT JOIN TRANSACCION t ON m.IDTransaccion = t.IDTransaccion
LEFT JOIN CLIENTE c ON t.IDCliente = c.IDCliente
LEFT JOIN PRODUCTO p ON m.IDProducto = p.IDProducto;
GO

```

```
-- VISTA_BUSCADOR_CLIENTES_RESUMEN
```

```
CREATE VIEW VISTA_BUSCADOR_CLIENTES_RESUMEN AS
```

```
SELECT
```

```

(c.PrimerNombre + ' ' + ISNULL(c.SegundoNombre,'') + ' ' + c.PrimerApellido + ' ' +
ISNULL(c.SegundoApellido,'')) AS NombreCompleto,

```

```
c.Numeroidentidad AS Identidad,
```

```
c.NumTel AS Telefono,
```

```
ISNULL(SUM(vp.TotalVenta),0) AS TotalCompras,
```

```
MAX(t.FechaEntrada) AS UltimaCompraFecha
```

```
FROM CLIENTE c
```

```
LEFT JOIN TRANSACCION t ON t.IDCliente = c.IDCliente
```

```

LEFT JOIN VENTA_PRODUCTO vp ON vp.IDTransaccion = t.IDTransaccion AND vp.Activo =
1

```

```
GROUP BY
```

```
c.PrimerNombre, c.SegundoNombre, c.PrimerApellido, c.SegundoApellido,
```

```
c.Numeroidentidad, c.NumTel, c.Activo;
```

```
GO
```

```
-- VISTA_BUSCADOR_FACTURAS_RESUMEN
```

```
CREATE VIEW VISTA_BUSCADOR_FACTURAS_RESUMEN AS
```

```
SELECT
```

```
'F-' + RIGHT('000000' + CAST(t.IDTransaccion AS VARCHAR(10)),6) AS NumeroFactura,
```

```
t.FechaEntrada,
```

```

(c.PrimerNombre + '' + ISNULL(c.SegundoNombre,'') + '' + c.PrimerApellido + '' +
ISNULL(c.SegundoApellido,'')) AS Cliente,

mp.NombreMetodo AS MetodoPago,

tt.NombreTipo AS TipoTransaccion,

ISNULL(SUM(vp.TotalVenta),0) AS MontoTotal,

COUNT(vp.IDVentaProducto) AS Lineas

FROM TRANSACCION t

LEFT JOIN CLIENTE c ON t.IDCliente = c.IDCliente

LEFT JOIN METODO_PAGO mp ON t.IDMetodoPago = mp.IDMetodoPago

LEFT JOIN TIPO_TRANSACCION tt ON t.IDTipoTransaccion = tt.IDTipoTransaccion

LEFT JOIN VENTA_PRODUCTO vp ON vp.IDTransaccion = t.IDTransaccion AND vp.Activo =
1

GROUP BY

t.IDTransaccion, t.FechaEntrada,

c.PrimerNombre, c.SegundoNombre, c.PrimerApellido, c.SegundoApellido,

mp.NombreMetodo, tt.NombreTipo, t.Activo;

GO

```

```

-- VISTA_TRANSACCIONES_DETALLE_EXPORTABLE

```

```

CREATE VIEW VISTA_TRANSACCIONES_DETALLE_EXPORTABLE AS

```

```

SELECT

```

```

'T-' + RIGHT('000000' + CAST(t.IDTransaccion AS VARCHAR(10)),6) AS
NumeroTransaccion,

```

```

CONVERT(VARCHAR(16), t.FechaEntrada, 120) AS Fecha,

```

```

(c.PrimerNombre + '' + ISNULL(c.SegundoNombre,'') + '' + c.PrimerApellido + '' +
ISNULL(c.SegundoApellido,'')) AS Cliente,

```

```

c.NumTel AS Telefono,

```

```

tt.NombreTipo AS TipoTransaccion,

```

```

mp.NombreMetodo AS MetodoPago,

```

```

ISNULL(SUM(vp.TotalVenta),0) AS MontoTotal,

```

```

STUFF((

```

```

SELECT ';' + p2.Nombre + ' x ' + CAST(vp2.CantidadVendida AS VARCHAR(20))
FROM VENTA_PRODUCTO vp2
LEFT JOIN PRODUCTO p2 ON vp2.IDProducto = p2.IDProducto
WHERE vp2.IDTransaccion = t.IDTransaccion AND vp2.Activo = 1
FOR XML PATH(''), TYPE).value(';', 'NVARCHAR(MAX)'),1,2,') AS ProductosResumen,
CASE WHEN EXISTS(SELECT 1 FROM MAQUILA_SEMILLA m WHERE m.IDTransaccion =
t.IDTransaccion) THEN 'Sí' ELSE 'No' END AS TieneMaquila
FROM TRANSACCION t
LEFT JOIN CLIENTE c ON t.IDCliente = c.IDCliente
LEFT JOIN TIPO_TRANSACCION tt ON t.IDTipoTransaccion = tt.IDTipoTransaccion
LEFT JOIN METODO_PAGO mp ON t.IDMetodoPago = mp.IDMetodoPago
LEFT JOIN VENTA_PRODUCTO vp ON vp.IDTransaccion = t.IDTransaccion AND vp.Activo =
1
GROUP BY
t.IDTransaccion, t.FechaEntrada,
c.PrimerNombre, c.SegundoNombre, c.PrimerApellido, c.SegundoApellido,
c.NumTel, tt.NombreTipo, mp.NombreMetodo, t.Activo;
GO

```

```

-- VISTA_VENTAS_POR_PRODUCTO_PERIODICA
CREATE VIEW VISTA_VENTAS_POR_PRODUCTO_PERIODICA AS
SELECT
p.Nombre AS Producto,
p.Categoria,
SUM(vp.CantidadVendida) AS CantidadVendidaEnPeriodo,
SUM(vp.TotalVenta) AS MontoTotalEnPeriodo,
CASE WHEN SUM(vp.CantidadVendida) = 0 THEN 0
ELSE ROUND(CAST(SUM(vp.CantidadVendida) AS DECIMAL(18,4)) /
NULLIF(COUNT(DISTINCT vp.IDTransaccion),0),2) END AS
UnidadesPorTransaccionPromedio,
RANK() OVER (ORDER BY SUM(vp.CantidadVendida) DESC) AS Ranking

```

```

FROM VENTA_PRODUCTO vp
INNER JOIN PRODUCTO p ON vp.IDProducto = p.IDProducto
WHERE vp.Activo = 1
GROUP BY p.Nombre, p.Categoria;
GO

```

```

-- VISTA_MOVIMIENTOS_POR_PRODUCTO_AUDITORIA
CREATE VIEW VISTA_MOVIMIENTOS_POR_PRODUCTO_AUDITORIA AS
SELECT
    CONVERT(VARCHAR(19), m.FechaMovimiento, 120) AS FechaMovimiento,
    p.Nombre AS Producto,
    tm.NombreMovimiento AS TipoMovimiento,
    m.CantidadMovida,
    ISNULL(m.Descripcion, '') AS Descripcion,
    CASE WHEN m.IDTransaccion IS NOT NULL THEN 'T-' + RIGHT('000000' +
    CAST(m.IDTransaccion AS VARCHAR(10)),6) ELSE NULL END AS TransaccionRef,
    NULL AS Usuario
FROM MOVIMIENTO_PRODUCTO m
LEFT JOIN PRODUCTO p ON m.IDProducto = p.IDProducto
LEFT JOIN TIPO_MOVIMIENTO tm ON m.IDTipoMovimiento = tm.IDTipoMovimiento;
GO

```

```

-- VISTA_INVENTARIO_VALORIZADO
CREATE VIEW VISTA_INVENTARIO_VALORIZADO AS
SELECT
    p.Nombre AS Producto,
    p.Categoria,
    p.Cantidad AS StockActual,
    p.PrecioUnitario,
    ROUND(p.Cantidad * p.PrecioUnitario,2) AS ValorInventario

```

FROM PRODUCTO p

GO

-- VISTA_MAQUILAS_PENDIENTES_Y_HISTORICO

CREATE VIEW VISTA_MAQUILAS_PENDIENTES_Y_HISTORICO AS

SELECT

'M-' + RIGHT('000000' + CAST(m.IDMaquila AS VARCHAR(10)),6) AS NumeroMaquila,

(c.PrimerNombre + ' ' + ISNULL(c.SegundoNombre,'') + ' ' + c.PrimerApellido + ' ' +
ISNULL(c.SegundoApellido,'')) AS Cliente,

m.OrigemSemilla,

ISNULL(p.Nombre,'(Sin producto)') AS Producto,

m.CantidadMaquilada,

m.PrecioPorUnidad,

ROUND(m.CantidadMaquilada * m.PrecioPorUnidad,2) AS MontoMaquila,

m.FechaInicio,

m.FechaEntrega,

DATEDIFF(DAY, GETDATE(), m.FechaEntrega) AS DiasRestantes,

CASE

WHEN GETDATE() <= m.FechaEntrega THEN 'En proceso'

WHEN GETDATE() > m.FechaEntrega THEN 'Entregada o retrasada'

ELSE 'Desconocido'

END AS Estado

FROM MAQUILA_SEMILLA m

LEFT JOIN TRANSACCION t ON m.IDTransaccion = t.IDTransaccion

LEFT JOIN CLIENTE c ON t.IDCliente = c.IDCliente

LEFT JOIN PRODUCTO p ON m.IDProducto = p.IDProducto;

GO

-- VISTA_PROVEEDORES_PRODUCTOS

CREATE VIEW VISTA_PROVEEDORES_PRODUCTOS AS


```

SELECT

pr.NombreProveedor AS Proveedor,

pr.TelefonoProveedor AS TelefonoProveedor,

pr.Activo AS ProveedorActivo,

ISNULL(

    STUFF((

        SELECT ';' + p2.Nombre

        FROM PRODUCTO p2

        WHERE p2.IDProveedor = pr.IDProveedor

        FOR XML PATH(''), TYPE).value(';', 'VARCHAR(MAX)'),1,2,''), '') AS
ProductosQueSuministra,

ISNULL((

    SELECT SUM(p3.Cantidad) FROM PRODUCTO p3 WHERE p3.IDProveedor =
pr.IDProveedor AND p3.Activo = 1

),0) AS StockTotalPorProveedor

FROM PROVEEDOR pr;

GO

-- VISTA_CLIENTES_TOP_Y_SEGMENTACION

CREATE VIEW VISTA_CLIENTES_TOP_Y_SEGMENTACION AS

SELECT

(c.PrimerNombre + '' + ISNULL(c.SegundoNombre,'') + '' + c.PrimerApellido + '' +
ISNULL(c.SegundoApellido,'')) AS Cliente,

ISNULL(SUM(vp.TotalVenta),0) AS TotalCompradoPeriodo,

COUNT(DISTINCT t.IDTransaccion) AS FrecuenciaCompras,

MAX(t.FechaEntrada) AS UltimaCompra

FROM CLIENTE c

LEFT JOIN TRANSACCION t ON t.IDCliente = c.IDCliente

LEFT JOIN VENTA_PRODUCTO vp ON vp.IDTransaccion = t.IDTransaccion AND vp.Activo =
1

GROUP BY c.PrimerNombre, c.SegundoNombre, c.PrimerApellido, c.SegundoApellido;

```

GO

-- 7. USUARIO TEMPORAL CON FECHA REAL

```
CREATE TABLE USUARIO_TEMPORAL_CONTROL (  
    IDUsuario INT PRIMARY KEY,  
    FechaCreacion DATETIME NOT NULL DEFAULT GETDATE(),  
    FechaExpiracion DATETIME NOT NULL  
);
```

GO

```
INSERT INTO USUARIO_TEMPORAL_CONTROL (IDUsuario, FechaExpiracion)  
SELECT IDUsuario, DATEADD(HOUR,1,GETDATE())  
FROM USUARIO  
WHERE Numeroidentidad='0000000000000';
```

GO

-- 8. PROCEDIMIENTOS PARA DESACTIVAR Y AUTENTICAR

```
CREATE OR ALTER PROCEDURE DesactivarUsuarioTemporal  
AS  
BEGIN  
    SET NOCOUNT ON;  
    UPDATE U  
    SET U.Activo=0  
    FROM USUARIO U  
    INNER JOIN USUARIO_TEMPORAL_CONTROL C ON U.IDUsuario=C.IDUsuario  
    WHERE U.Numeroidentidad='0000000000000'  
    AND U.Activo=1  
    AND GETDATE() > C.FechaExpiracion;  
END;
```

GO

CREATE OR ALTER PROCEDURE AutenticarUsuario

 @Numeroidentidad VARCHAR(15),

 @Clave VARCHAR(100)

AS

BEGIN

 EXEC DesactivarUsuarioTemporal;

 SELECT

 U.IDUsuario,

 U.Numeroidentidad,

 U.PrimerNombre,

 U.SegundoNombre,

 U.PrimerApellido,

 U.SegundoApellido,

 U.Clave,

 U.IDRol,

 U.Activo,

 C.FechaCreacion,

 C.FechaExpiracion

 FROM USUARIO U

 LEFT JOIN USUARIO_TEMPORAL_CONTROL C ON U.IDUsuario=C.IDUsuario

 WHERE U.Numeroidentidad=@Numeroidentidad AND U.Clave=@Clave;

END;

GO

5. Instalación del sistema

5.1 Requisitos previos

- Windows 8, 10 o 11
- SQL Server Express o superior
- .NET Framework 4+
- Permisos de administrador para instalar dependencias

5.2 Procedimiento

1. Copiar la carpeta del proyecto a la máquina destino.
2. Crear la base de datos (si corresponde).
3. Configurar la cadena de conexión en el archivo Program.cs o la clase de conexión.
4. Ejecutar **INSAVA01.exe** desde bin/Debug.

6. Configuración técnica

6.1 Cadena de conexión

```
public ClaseConexion(string host = null, string nombreDB = "SISTEMASEMILLA")
{
    // Si no se pasa un host, usar el servidor local
    _host = host ?? ".";
    _nombreDB = nombreDB;
    _conexion = new
    SqlConnection($"Server={_host};Database={_nombreDB};Integrated
    Security=True;Encrypt=False;");
}
```

6.2 Configuración de reportes

Los reportes usan:

- Archivos .rdlc
- DataSets tipados
- Conexión local al formulario ReportViewer

6.3 Configuración de roles y usuarios

Estos se almacenan en BD y permiten:

- Acceso restringido a módulos
- Seguridad en operaciones críticas

7. Descripción técnica por módulos

Cada módulo corresponde a un formulario WinForms con lógica asociada.

7.1 Módulo de Clientes

Archivo: frmClientes.cs

Funciones internas:

- Insert, Update, Delete desde SQL
- Validación de campos
- Carga de datos con DataGridView
- Búsqueda por coincidencias

7.2 Módulo de Proveedores

Archivo: frmProveedores.cs

- CRUD completo
- Listado dinámico
- Control de integridad (si un proveedor tiene compras no se puede eliminar)

7.3 Módulo de Productos

Archivo: frmProductos.cs

Funciones técnicas:

- Control de stock
- Actualización automática desde compras/ventas
- Validación de precios
- Carga de imagen (si aplica)

7.4 Módulo de Compras

Archivo: frmCompras.cs

Lógica:

- Inserción en tabla Compras
- Inserción automática en DetalleCompra

- Incremento de inventario
- Registro en Kardex → Movimiento tipo **Entrada**

7.5 Módulo de Ventas

Archivo: frmVentas.cs

Lógica interna:

- Descuento de stock
- Inserción en Ventas y DetalleVenta
- Movimiento tipo **Salida** en Kardex
- Generación de factura en ReportViewer

7.6 Módulo de Kardex

Archivo: frmKardex.cs

Registro automático de movimientos:

- Entrada por compra
- Salida por venta
- Cálculo de existencias actualizadas

7.7 Módulo de Usuarios

Lógica clave:

- Hash o almacenamiento simple de contraseñas
- Validación de acceso
- Asignación de permisos

7.8 Configuración de Empresa

Datos utilizados para:

- Facturación
- Reportes
- Encabezados oficiales

7.9 Descripción de Clases del Sistema INSAVA01

A continuación se describen las clases principales del sistema y la función que cumple cada una dentro de la lógica de negocio y la estructura interna del software.

1. Clase Conexion

Es la clase encargada de administrar la conexión entre el sistema y la base de datos.

Proporciona métodos para abrir, cerrar y ejecutar consultas SQL.

Todas las operaciones de CRUD dependen de ella.

Función principal: permitir comunicación estable y segura con SQL Server.

2. Clase Producto

Representa un producto del inventario.

Incluye atributos como: ID, nombre, descripción, precio, unidad, categoría, marca y stock.

Función principal: servir como modelo para los productos que se registran, compran, venden y reportan.

3. Clase Cliente

Contiene la información personal y de contacto de los clientes.

Función principal: permitir registrar ventas y generar facturación asociada al cliente correspondiente.

4. Clase Proveedor

Modelo de datos que almacena la información de los proveedores.

Función principal: permitir registrar compras y mantener un historial de transacciones con cada proveedor.

5. Clase Usuario

Guarda los datos de los usuarios del sistema, como nombre, contraseña, rol o nivel de permisos.

Función principal: controlar el acceso al sistema y registrar quién realiza cada operación.

6. Clase Venta

Representa la estructura de una venta completa: fecha, cliente, usuario que la registró, subtotal, impuestos y total.

Función principal: manejar el proceso de salida de inventario y registrar cada transacción de venta.

7. Clase DetalleVenta

Describe los productos incluidos en cada venta: cantidad, precio unitario, descuento y subtotal.

Función principal: permitir que una venta contenga múltiples productos y actualizar el inventario de forma precisa.

8. Clase Compra

Define los datos generales de una compra: proveedor, fecha, usuario, subtotal, impuestos y total.

Función principal: registrar las entradas de inventario y generar un historial de adquisiciones.

9. Clase DetalleCompra

Lista los productos adquiridos dentro de una compra, incluyendo cantidades y costos.

Función principal: incrementar el inventario por cada producto comprado y registrar cada movimiento.

10. Clase Kardex

Registra cada entrada y salida del inventario de forma cronológica.
Se actualiza automáticamente según ventas y compras.

Función principal: llevar un control detallado del inventario y soportar auditorías.

11. Clase Categoria

Clasifica los productos en grupos organizados, como “Materiales”, “Herramientas”, etc.

Función principal: facilitar el control, búsqueda y reportes del inventario categorizado.

12. Clase Marca

Representa la marca o fabricante de cada producto.

Función principal: apoyar la clasificación y permitir reportes basados en la marca del producto.

13. Clase UnidadMedida

Define la unidad en que se controla un producto: piezas, metros, cajas, litros, etc.

Función principal: estandarizar el manejo del inventario y evitar errores en cantidades.

14. Clase Inventario

Contiene las existencias actuales de cada producto, usualmente calculadas desde kardex.

Función principal: proporcionar el valor real de stock disponible para ventas, compras y reportes.

15. Clase Reportes

Maneja la comunicación entre el sistema y los reportes RDLC.

Carga datos, parámetros y estructuras para facturas, inventarios y otros reportes.

Función principal: generar documentos imprimibles y reportes profesionales.

8. Seguridad

8.1 Seguridad de acceso

- Usuarios autenticados
- Restricción por rol (si está implementado)

8.2 Seguridad de BD

- Respaldos regulares
- Carpeta bin protegida
- No modificar las tablas manualmente

9. Mantenimiento técnico

9.1 Tareas periódicas

- Depurar registros antiguos
- Revisions de stock
- Actualizar librerías .NET
- Optimizar base de datos (índices, compactación)

9.2 Respaldo

- Copias de la carpeta del sistema
- Copias de la base de datos SQL Server (*.bak*)

10. Troubleshooting técnico

Problema	Causa técnica	Solución
El exe no abre	Falta DLL o no coincide Framework	Reinstalar dependencias
No carga reportes	Falta ReportViewer	Instalar runtime
Inventario no actualiza	Error en transacción SQL	Verificar triggers y consultas